TECNOLÓGICO DE COSTA RICA

LISTA DIGITAL DE CONTACTOS

Documentación del Proyecto

Curso: Taller de Programación

Profesor: William Mata Rodríguez

Estudiante: Frankie Cheng Zheng

Fecha: Octubre de 2026

Contenido

1. Enunciado del Proyecto 3
1.1. Objetivos 3
1.2. Definición 4
1.3. Requerimientos 5
2. Investigación de temas. 6
2.1. Herramientas de depuración (debugger) 6
2.2. Funciones del depurador de IDLE 7
2.3. Importancia de la documentación de software. 8
2.4. Expresiones regulares. 9
2.5. Generación de archivos PDF. 10
2.6. Control de versiones y trabajo colaborativo. 11
2.7. Buenas prácticas de programación. 12
3. Uso de inteligencia artificial en el proyecto. 13
4. Conclusiones 17
Link de github 18
5. Estadística del tiempo utilizado 18
6. Lista de revisión del proyecto y análisis de resultados 19

1. Enunciado del Proyecto

1.1. Objetivos

Objetivo general

Desarrollar un programa en Python que permita registrar, consultar, modificar y eliminar información de contactos, aplicando los conocimientos adquiridos durante el curso Taller de Programación.

Objetivos específicos

- Aplicar estructuras de datos como listas, tuplas y cadenas de texto para almacenar y administrar la información.
- Utilizar funciones, ciclos y estructuras condicionales para organizar el funcionamiento del programa.
- Implementar validaciones para asegurar que los datos ingresados cumplan con los requisitos establecidos.
- Mantener la integridad referencial entre los contactos y los grupos a los que pertenecen.
- Implementar operaciones para agregar, consultar, modificar y eliminar información.
- Generar un archivo PDF con la lista de contactos registrados.
- Utilizar herramientas de depuración y control de versiones durante el desarrollo del proyecto.
- Elaborar un manual de usuario y la documentación correspondiente al programa.

1.2. Definición

El proyecto consiste en desarrollar un programa en Python llamado **Lista Digital de Contactos**, cuyo propósito es permitir al usuario registrar, organizar y administrar información de contactos mediante un menú de opciones.

El programa permite registrar y administrar áreas, configurar valores predeterminados, registrar contactos, organizar contactos en grupos y consultar la lista de contactos mediante diferentes filtros.

Además, incluye la posibilidad de generar un archivo PDF con la lista de contactos, consultar un manual de usuario desde la opción Ayuda y visualizar información general del programa mediante la opción Acerca de.

El sistema se desarrolla utilizando los conocimientos adquiridos durante el curso Taller de Programación y aplica estructuras de datos, funciones, estructuras condicionales y ciclos para implementar sus funcionalidades.

1.3. Requerimientos

El programa Lista Digital de Contactos utiliza una interfaz de texto que permite al usuario interactuar mediante el teclado. Al ejecutarlo, se muestra un menú principal con diferentes opciones para administrar la información de los contactos.

Las funcionalidades principales son:

1. Registrar áreas: permite agregar, consultar, modificar y eliminar áreas telefónicas. Cada área contiene un código y un nombre.

2. Configuración de lista de contactos: permite establecer el área y el tipo de teléfono que se utilizarán como valores predeterminados al registrar contactos.

3. Registrar contactos: permite agregar, consultar, modificar y eliminar contactos. Cada contacto contiene información como teléfono, área, tipo de teléfono, nombre, correo electrónico, dirección, fecha de nacimiento, pasatiempos y notas.

4. Administrar grupos de contactos: permite crear, modificar y eliminar grupos, además de agregar o eliminar contactos de estos grupos.

5. Lista de contactos: permite consultar los contactos mediante filtros y generar un listado en formato PDF. También contempla el uso del comodín % para realizar búsquedas.

6. Ayuda: permite abrir el manual de usuario del programa en formato PDF.

7. Acerca de: muestra información general del programa, como su nombre, versión, fecha de creación y autor.

0. Fin: permite finalizar la ejecución del programa.

Además, el sistema debe validar los datos ingresados por el usuario y mantener la integridad referencial entre contactos y grupos. Por ejemplo, debe evitar que se elimine un contacto cuando todavía pertenece a un grupo.

Para desarrollar estas funcionalidades se utilizan estructuras de datos como listas y tuplas, funciones, ciclos, condiciones y la estructura match-case para controlar el menú principal.

2. Investigación de temas.

2.1. Herramientas de depuración (debugger)

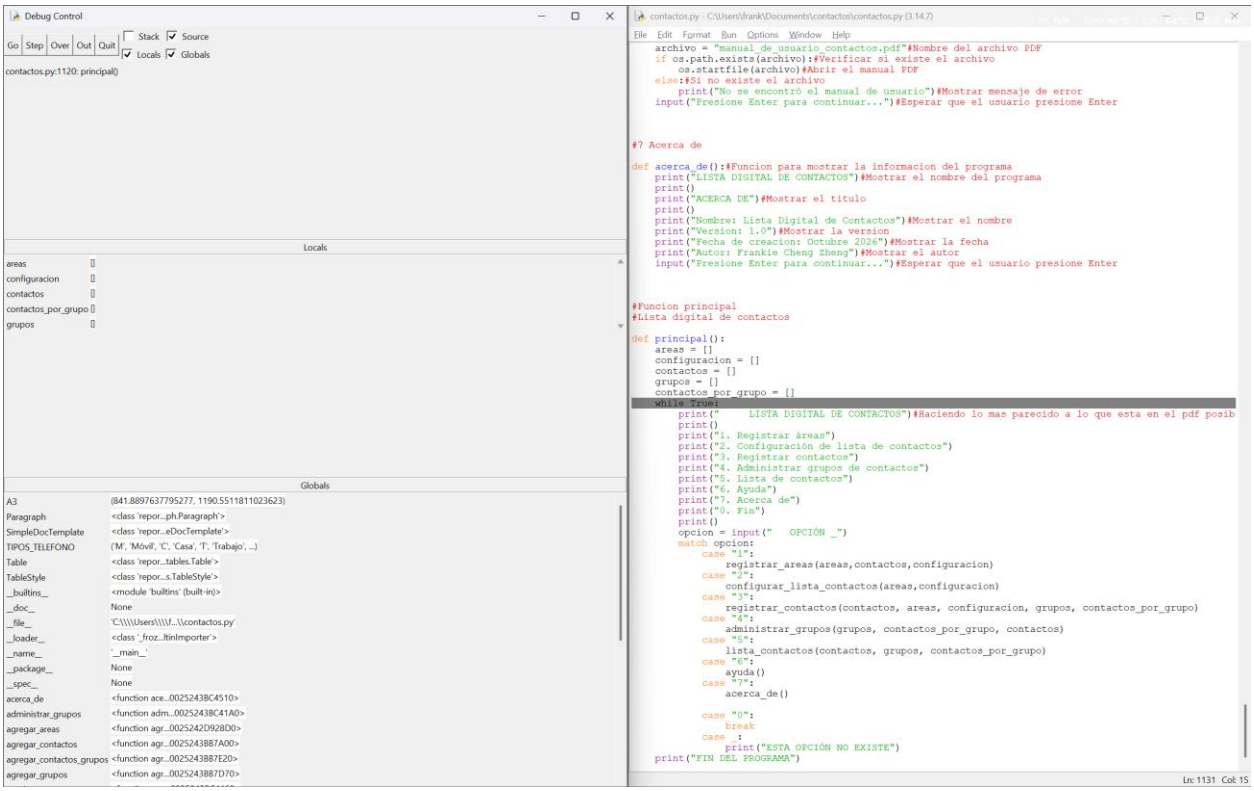


Figura 1. Uso del depurador de IDLE para observar las variables de la función principal del programa.

Un depurador (*debugger*) es una herramienta que permite analizar la ejecución de un programa paso a paso para identificar errores y comprender su funcionamiento. A diferencia de una ejecución normal, permite detener el programa, revisar las instrucciones que se ejecutan y observar los valores de las variables.

En este proyecto se utilizó el depurador integrado en IDLE, el entorno de desarrollo de Python. Se emplearon las opciones Step y Over para avanzar por las instrucciones y observar el funcionamiento del programa.

Durante la prueba se examinó la función principal(), donde se inicializan las listas areas, configuracion, contactos, grupos y contactos_por_grupo. Mediante la sección Locals del depurador se comprobó que estas variables comenzaban como listas vacías, lo cual corresponde al estado inicial del programa.

Esta herramienta resulta útil para detectar errores, verificar los cambios de las variables y comprender el flujo de ejecución de las funciones.

2.2. Funciones del depurador de IDLE

El entorno IDLE de Python incluye un depurador que permite controlar la ejecución del programa y examinar sus variables. Sus principales funciones son:

- **Go:** continúa la ejecución del programa hasta encontrar una pausa o finalizar.
- **Step:** avanza una instrucción y permite entrar en las funciones llamadas.
- **Over:** ejecuta la instrucción actual sin entrar en los detalles internos de las funciones llamadas.
- **Out:** continúa la ejecución hasta salir de la función actual.
- **Quit:** detiene la ejecución del programa en el depurador.
- **Locals:** muestra las variables locales de la función que se está ejecutando.
- **Globals:** muestra las variables globales disponibles.
- **Stack:** permite observar la secuencia de llamadas a funciones.
- **Source:** permite mostrar la ubicación de la ejecución en el código fuente.

Aplicación en el proyecto: Durante la depuración de la Lista Digital de Contactos se utilizaron principalmente Step, Over, Locals y Globals. Estas opciones permitieron observar la ejecución de la función principal y comprobar la inicialización de las listas utilizadas para almacenar la información del programa.

Python Software Foundation. (s. f.). *IDLE — Python editor and shell*. Python Documentation. https://docs.python.org/3/library/idle.html

2.3. Importancia de la documentación de software.

La documentación de software consiste en registrar y explicar información importante sobre el funcionamiento, desarrollo y uso de un programa. Su objetivo es facilitar la comprensión del sistema, tanto para los usuarios como para las personas encargadas de mantenerlo o modificarlo.

Existen dos tipos importantes de documentación:

- **Documentación interna:** se encuentra dentro del código fuente e incluye comentarios, nombres descriptivos de variables y funciones, y explicaciones de procesos.
- **Documentación externa:** se presenta en documentos independientes, como manuales de usuario, guías técnicas e informes del proyecto.

Importancia: una buena documentación facilita la identificación de errores, el mantenimiento del programa, la incorporación de nuevas funcionalidades y la comprensión del código.

Aplicación en el proyecto: en la Lista Digital de Contactos se utilizó documentación interna mediante comentarios que explican diferentes instrucciones y funciones de Python. También se elaboró un manual de usuario en formato PDF, con capturas de pantalla e instrucciones para utilizar las opciones del menú.

Además, se preparó la documentación del proyecto para registrar los temas investigados, las herramientas utilizadas, las pruebas realizadas y los resultados obtenidos.

Python Software Foundation. (s. f.). *PEP 257 – Docstring conventions*. Python Enhancement Proposals. https://peps.python.org/pep-0257/

2.4. Expresiones regulares.

Las expresiones regulares, también conocidas como *regex*, son patrones utilizados para buscar, comparar y validar cadenas de texto. Permiten comprobar si una cadena cumple con una estructura determinada, por ejemplo, un correo electrónico, un número telefónico o una fecha.

En Python, las expresiones regulares se pueden utilizar mediante el módulo re, que incluye funciones para comprobar patrones dentro de cadenas de texto.

Aplicación en la validación de correos electrónicos

Una expresión regular sencilla para comprobar la estructura básica de un correo electrónico es:

^[^\s@]+@[^\s@]+$

Este patrón comprueba que exista texto antes y después del símbolo @, que haya un único símbolo @ y que no existan espacios en blanco. Sin embargo, no garantiza que la dirección exista ni que cumpla todas las reglas posibles de los correos electrónicos.

Aplicación en el proyecto

En la Lista Digital de Contactos, la validación del correo electrónico se implementó mediante operaciones con cadenas de texto, sin utilizar directamente el módulo re.

El programa utiliza count("@") para comprobar que exista un único símbolo @, split("@") para separar las dos partes del correo y una condición para verificar que no contenga espacios.

Aunque no se utilizaron expresiones regulares en esta validación, podrían emplearse como una alternativa para comprobar la estructura del correo electrónico.

Otras posibles aplicaciones

Las expresiones regulares también podrían utilizarse para verificar que los números telefónicos contengan únicamente dígitos y para comprobar el formato de las fechas de nacimiento. No obstante, para verificar que una fecha exista realmente, seguiría siendo necesario utilizar una herramienta como datetime.

Python Software Foundation. (s. f.). *re — Regular expression operations*. Python Documentation. https://docs.python.org/3/library/re.html

2.5. Generación de archivos PDF.

El formato PDF (*Portable Document Format*) permite almacenar y compartir documentos conservando su presentación, independientemente del dispositivo o programa utilizado para abrirlos. Es útil para generar reportes, formularios, listas y otros documentos que necesitan mantener un formato organizado.

En Python existen bibliotecas que permiten crear archivos PDF automáticamente. Una de ellas es **ReportLab**, que ofrece herramientas para escribir texto, crear tablas y organizar información dentro de las páginas.

Aplicación en el proyecto

En la Lista Digital de Contactos se utilizó la biblioteca ReportLab para generar un archivo PDF con la información de los contactos registrados.

Esta funcionalidad corresponde a la opción **5. Lista de contactos** del menú principal. El programa permite consultar los contactos y generar un documento PDF con los resultados.

Para implementar esta función se utilizaron herramientas de ReportLab que permiten organizar la información en el documento. También se utilizó escape del módulo xml.sax.saxutils para tratar caracteres especiales presentes en los textos.

Importancia

La generación de PDF permite presentar la información de manera ordenada y facilita que el usuario pueda consultar, guardar o compartir la lista de contactos sin necesidad de ejecutar el programa de Python.

ReportLab. (s. f.). *ReportLab PDF library user guide*. https://docs.reportlab.com/reportlab/userguide/

2.6. Control de versiones y trabajo colaborativo.

El control de versiones es un sistema que permite registrar los cambios realizados en los archivos de un proyecto. Su importancia consiste en conservar un historial del desarrollo, identificar modificaciones y facilitar la recuperación de versiones anteriores cuando ocurre algún problema.

También es útil para el trabajo colaborativo, porque permite que varias personas participen en el desarrollo de un mismo proyecto y organicen sus cambios.

Tres herramientas relacionadas con el control de versiones

- **Git:** es un sistema de control de versiones que permite registrar los cambios del código mediante commits y consultar el historial del proyecto.
- **GitHub:** es una plataforma que permite almacenar repositorios Git en Internet, compartir proyectos y colaborar con otros desarrolladores.
- **GitLab:** es otra plataforma que permite alojar repositorios Git y ofrece herramientas para colaborar, revisar cambios y organizar proyectos de software.

Aplicación en el proyecto

Para desarrollar la Lista Digital de Contactos se utilizaron Git y GitHub. El código se guardó en un repositorio y se realizaron commits para registrar los avances del programa.

Mediante Git Bash se utilizaron comandos como git add para preparar los archivos, git commit para guardar los cambios en el historial y git push origin main para subirlos al repositorio de GitHub.

Estas herramientas permitieron mantener un respaldo del proyecto y conservar un historial de las modificaciones realizadas.

GitLab se investigó como una alternativa, pero no se utilizó directamente en el desarrollo del programa.

Buenas prácticas de control de versiones y trabajo colaborativo

Para trabajar con Git y GitHub es recomendable realizar commits frecuentes, escribir mensajes claros que expliquen los cambios y revisar el código antes de subirlo al repositorio.

Cuando varias personas trabajan en un mismo proyecto, también es importante utilizar ramas para desarrollar cambios sin afectar la versión principal, revisar las modificaciones antes de unirlas y mantener una buena comunicación para evitar conflictos.

En mi proyecto utilicé GitHub para guardar los avances y mantener un historial de los cambios realizados. Aunque el trabajo fue individual, estas prácticas también serían útiles si tuviera que desarrollar un programa en equipo.

Git. (s. f.). *Git documentation*. https://git-scm.com/doc

GitHub. (s. f.). *About GitHub and Git*. GitHub Docs. https://docs.github.com/en/get-started/start-your-journey/about-github-and-git

GitLab. (s. f.). *GitLab documentation*. https://docs.gitlab.com/

2.7. Buenas prácticas de programación.

Las buenas prácticas de programación son recomendaciones que ayudan a desarrollar código más organizado, comprensible y fácil de mantener. Su objetivo es reducir errores y facilitar que otras personas puedan entender o modificar un programa.

Entre las principales buenas prácticas se encuentran:

- **Utilizar nombres descriptivos:** asignar nombres claros a las variables y funciones para identificar su propósito.
- **Dividir el programa en funciones:** separar las tareas del programa para evitar que todo el código se encuentre en un solo bloque.
- **Validar los datos:** comprobar la información ingresada por el usuario para evitar errores.
- **Evitar código repetido:** reutilizar funciones cuando varias partes del programa necesitan realizar una misma tarea.
- **Agregar comentarios útiles:** explicar las partes del código que pueden resultar difíciles de comprender.
- **Realizar pruebas:** comprobar que las funcionalidades trabajen correctamente antes de entregar el programa.

Aplicación en el proyecto

En la Lista Digital de Contactos se organizaron las funcionalidades mediante diferentes funciones y un menú principal. Esto permitió separar las tareas de registrar áreas,

configurar el programa, registrar contactos, administrar grupos y generar la lista de contactos.

También se implementaron validaciones para controlar los datos ingresados por el usuario y se utilizaron nombres de variables relacionados con la información almacenada, como areas, contactos y grupos.

Además, se realizaron pruebas de las opciones del menú para comprobar su funcionamiento y se utilizó el depurador de IDLE para observar la ejecución del programa.

Van Rossum, G., Warsaw, B., & Coghlan, N. (2001). *PEP 8 – Style guide for Python code*. Python Enhancement Proposals. https://peps.python.org/pep-0008/

3. Uso de inteligencia artificial en el proyecto.

Objetivo del uso	Aprender cómo abrir un archivo PDF desde un programa de Python.
Herramienta utilizada	ChatGPT.
Prompt o pregunta	Consulté cómo podía hacer que la opción Ayuda abriera el manual de usuario en PDF desde el programa. (Resumen de la consulta).
Respuesta	me explicó que podía utilizar os.startfile() para abrir un archivo PDF desde Python.
¿Cómo usó o adaptó la respuesta?	Utilicé esa función en la opción 6 del menú principal para que el usuario pudiera abrir el manual sin tener que buscarlo en la carpeta.
Reflexión crítica	Al principio no sabía cómo abrir un PDF desde Python. La explicación me ayudó a entenderlo, pero tuve que comprobar que el archivo estuviera en la ubicación correcta.
Otros	Probé la opción Ayuda desde el menú principal para verificar que el manual se abriera.

Objetivo del uso	Recibir ayuda para generar un archivo PDF con la lista de contactos utilizando la biblioteca ReportLab en Python.
Herramienta utilizada	ChatGPT .
Prompt o pregunta	Consulté cómo podía generar un archivo PDF con los contactos registrados utilizando ReportLab. (Resumen de las consultas).
Respuesta	Se explicaron las funciones de ReportLab y se proporcionaron ejemplos de código para crear documentos PDF desde Python.
¿Cómo usó o adaptó la respuesta?	Utilicé la ayuda para desarrollar la opción 5 del menú principal, que permite generar un PDF con la información de los contactos.
Reflexión crítica	No sabía cómo generar archivos PDF con Python, por lo que necesité ayuda para entender ReportLab. Después tuve que probar el programa y revisar que la información apareciera correctamente.
Otros	Realicé pruebas para comprobar que el archivo PDF se generara con los datos registrados.

Objetivo del uso	Aprender a validar los datos que ingresa el usuario utilizando isdigit() y strip().
Herramienta utilizada	ChatGPT.
Prompt o pregunta	Consulté cómo comprobar que un dato tuviera solamente números y cómo evitar que el usuario ingresara nombres con solo espacios. (Resumen de las consultas).
Respuesta	ChatGPT me explicó que isdigit() sirve para comprobar si un texto contiene únicamente dígitos y que strip() permite quitar los espacios al principio y al final de un texto.

¿Cómo usó o adaptó la respuesta?	Utilicé isdigit() para validar los teléfonos y códigos de área. También utilicé strip() para evitar que se registraran nombres de contactos, áreas o grupos que contuvieran solamente espacios.
Reflexión crítica	No conocía bien estas funciones y al principio tenía dudas sobre cómo utilizarlas. Con las explicaciones pude entender para qué servían y aplicarlas en diferentes partes del programa.
Otros	Probé las validaciones ingresando letras donde se pedían números y espacios donde se solicitaban nombres.

Objetivo del uso	Aprender a validar fechas de nacimiento en Python.
Herramienta utilizada	ChatGPT.
Prompt o pregunta	Consultas sobre cómo comprobar que una fecha de nacimiento fuera válida y tuviera el formato dd/mm/aaaa. (Resumen de las consultas).
Respuesta	ChatGPT me explicó cómo utilizar datetime.strptime() para comprobar fechas y cómo manejar los errores cuando se ingresa una fecha incorrecta.
¿Cómo usó o adaptó la respuesta?	Utilicé esta ayuda en las opciones de agregar y modificar contactos. El programa comprueba el formato de la fecha y utiliza datetime.strptime() para verificar que sea válida.
Reflexión crítica	Esta parte me costó porque no conocía bien el módulo datetime. Las explicaciones me ayudaron a entender cómo validar una fecha y evitar que el programa aceptara fechas incorrectas.
Otros	El programa también contempla las fechas desconocidas y el caso especial del año 0000, según los requisitos del proyecto.

Objetivo del uso	Entender cómo modificar y eliminar elementos de las listas en Python.
Herramienta utilizada	ChatGPT

Prompt o pregunta	Consultas sobre cómo utilizar clear(), pop() y remove() para modificar los datos guardados en las listas del programa. (Resumen de las consultas).
Respuesta	ChatGPT me explicó que clear() elimina todos los elementos de una lista, pop() permite eliminar un elemento por su posición y remove() elimina un elemento por su valor.
¿Cómo usó o adaptó la respuesta?	Utilicé clear() para reemplazar la configuración anterior, pop() para eliminar áreas o grupos y remove() para eliminar contactos registrados o contactos asociados a un grupo.
Reflexión crítica	Al principio me confundía entre estas funciones porque todas permiten quitar elementos. Las explicaciones me ayudaron a entender cuándo utilizar cada una según lo que necesitaba hacer.
Otros	Revisé su funcionamiento en las opciones de configuración y eliminación del programa.

Objetivo del uso	Aprender a utilizar funciones de texto para buscar y filtrar contactos.
Herramienta utilizada	ChatGPT.
Prompt o pregunta	Consulté cómo hacer que el programa buscara contactos utilizando filtros y cómo trabajar con lower(), find() y split(). (Resumen de las consultas).
Respuesta	ChatGPT me explicó que lower() convierte el texto a minúsculas, find() busca una parte del texto y split() permite dividir una cadena utilizando un separador.
¿Cómo usó o adaptó la respuesta?	Utilicé estas funciones en cumple_filtro(), que permite buscar contactos utilizando el símbolo %. Con lower() la búsqueda no distingue entre mayúsculas y minúsculas, mientras que split() y find() ayudan a comprobar si el texto coincide con el filtro.
Reflexión crítica	Esta parte fue una de las más difíciles porque tenía que combinar varias funciones para que los filtros funcionaran

	correctamente. La ayuda me permitió entender mejor cómo se realiza la búsqueda de texto.
Otros	Revisé los filtros de la opción 5, Lista de contactos, para comprobar cómo se aplican a los datos registrados.

4. Conclusiones

4.1. Problemas encontrados y soluciones

Una de las cosas que más me costó durante el proyecto fue la indentación del código, ya que en varias ocasiones una instrucción mal acomodada provocaba errores o hacía que el programa no funcionara como esperaba. También tuve dificultades con las validaciones, porque tenía que revisar muchas condiciones para evitar que el usuario ingresara datos incorrectos.

Otro problema que encontré fue que algunas opciones, si se seguían exactamente como aparecían en el PDF del proyecto, podían dejar al usuario sin una forma clara de regresar. Por eso decidí agregar la opción de cancelar con la letra C en diferentes partes del programa.

Para resolver estos problemas fui revisando el código, haciendo pruebas y corrigiendo los errores que encontraba.

4.2. Aprendizajes obtenidos

Durante el desarrollo de este proyecto aprendí bastante sobre cómo organizar un programa utilizando funciones, listas y diferentes opciones de menú. También pude practicar temas que habíamos visto en clase y conocer algunas funciones nuevas de Python.

Aprendí la importancia de realizar validaciones para evitar errores al ingresar datos y de utilizar el depurador de IDLE para revisar el funcionamiento del programa. Además, aprendí a generar archivos PDF con Python y a utilizar GitHub para guardar los avances del proyecto.

En general, este proyecto me permitió entender mejor cómo funciona un programa más grande y la importancia de revisar los detalles para que las diferentes opciones funcionen correctamente.

4.3. ¿Qué haría diferente en un próximo proyecto?

Si tuviera que realizar otro proyecto parecido, intentaría organizar mejor el código desde el principio y probar cada función antes de continuar con la siguiente. Esto me ayudaría a encontrar los errores más rápido y evitar que se acumularan.

También intentaría reducir las validaciones repetidas y organizar mejor el tiempo dedicado a cada parte del proyecto. De esta manera, podría dedicar más tiempo a las pruebas finales y a revisar que todo funcione correctamente antes de entregar el programa.

Link de github

https://github.com/Frankie-AFK/contactos

5. Estadística del tiempo utilizado

Actividad Realizada	Horas
Análisis del problema	4
Diseño de algoritmos	5
Investigación de funciones y bibliotecas	5
Programación	18
Corrección de errores y validaciones	8
Pruebas del programa	4
Elaboración del manual de usuario	3
Elaboración de documentación del proyecto	3
TOTAL	50

6. Lista de revisión del proyecto y análisis de resultados

Concepto	Puntos	Avance 100/%/0	Puntos obtenidos	Análisis de resultados
Registrar áreas Agregar 3P Consultar 1P Modificar 3P Eliminar 3P	10	100 %	10	
Configuración	5	100 %	5	Para utilizar esta opción primero se deben registrar áreas.
Registrar contactos Agregar 6P Consultar 2P Modificar 6P Eliminar 6P	20	100 %	20	Para registrar contactos primero deben existir áreas registradas.
Administrar grupos de contactos Agregar grupos 2P Agregar contactos 4P Modificar grupos 6P Eliminar grupos 5P Eliminar contactos de los grupos 3P	20	100 %	20	Para agregar contactos a un grupo primero deben existir contactos registrados.
Lista de contactos Uso de comodín % 8P No usa comodines (%) 2P	10	100 %	10	
Emitir listado de contactos en formato PDF	10	100 %	10	
Validación de datos de entrada	10	100 %	10	
Validación de integridad referencial	10	100 %	10	
Ayuda en el programa: Manual de Usuario desplegado en la pantalla cuando el usuario solicita la opción de ayuda	5	100 %	5	
TOTAL	100	100 %	100	
Partes desarrolladas adicionalmente				Agregué la opción de cancelar con la letra C en diferentes operaciones, permitiendo regresar sin completarlas. Esta mejora surgió durante las pruebas para facilitar el uso del programa.